

01_local_models_serving

Local LLM Serving for Coding + CLI-Agent Work on RTX 5090 (32 GB, Blackwell sm_120)

Target hardware: Gigabyte RTX 5090 AORUS MASTER — 32 GB GDDR7, ~1.79 TB/s memory bandwidth, Blackwell architecture, CUDA compute capability **sm_120**. Linux host, powerful CPU/RAM. **Workload:** 2–3 OpenCode / CLI-agent instances × 3–4 subagents = **6–12 concurrent decode sequences**, coding + agentic tool-calling, 16k–32k context, target >30 tok/s per stream. **Report date / sources verified:** 2026-07-06.

Honesty / anti-bluff notes (read first). 1. Almost no *measured* Qwen2.5-Coder-32B tok/s number on a 5090 exists in a primary source; the tok/s figures below are a mix of one measured leaderboard (awesomeagents), vendor/estimate pages (modelfit explicitly says “estimates from memory bandwidth, not measured benchmarks”), and Blackwell-family (RTX PRO 6000, same sm_120) benchmarks. Each is tagged. 2. The web-search environment surfaced several **future/aggregator model names** (GLM-4.7, GLM-5, Qwen3.5/3.6, DeepSeek V4) on low-quality sites with no primary benchmark. These are marked **UNCONFIRMED** and are NOT used as recommendation anchors. Recommendations rest on models with published model cards + benchmarks: Qwen2.5-Coder-32B, Qwen3-32B, Qwen3-Coder-30B-A3B, Devstral-Small-2507, GLM-4.5-Air. 3. KV-cache figures are computed from published model architecture (formula shown), not copied from aggregators (one aggregator quoted a 32 GB FP16 figure for a “32B” that does not match GQA math — corrected below).

1. Executive recommendation

Question	Recommendation
Primary model (does both coding + agentic tool-use)	Qwen3-Coder-30B-A3B-Instruct (MoE, 30.5B total / 3.3B active). Fast (~140 tok/s single-stream, cited), tiny KV cache (MoE + 4 KV heads) → best concurrency-per-GB, 256K native

Question	Recommendation
	context, tool-calling fixes shipped in Unsloth GGUFs.
(a) Heavy pure-coding quality	Qwen2.5-Coder-32B-Instruct (dense) — best measured code benchmarks (HumanEval 88.4, Aider 73.7) but slow (~45–58 tok/s) and expensive KV; use when quality > throughput and concurrency is low.
(b) CLI-agent / tool-use focused	Devstral-Small-2507 (24B dense, SWE-bench Verified 53.6%, #1 open agentic model July 2025, 128K ctx) OR Qwen3-Coder-30B-A3B . Devstral is purpose-built for coding <i>agents</i> (multi-file edits, codebase exploration).
Serving stack for 6–12 concurrent agents	vLLM (built from source for sm_120) with an AWQ 4-bit checkpoint — continuous batching + PagedAttention give by far the best aggregate throughput (~8× Ollama at load, cited). Fallback / easiest: llama-server (llama.cpp) with --parallel --cont-batching -fa — simpler on Blackwell, good for moderate concurrency.
Avoid for this GPU (now)	SGLang (FP8-blockwise unsupported on sm_120; kernel gaps), TensorRT-LLM (NVFP4 not wired for SM120 in trtllm-gen), Ollama for concurrency (collapses at 16+ users). GLM-4.5-Air / 70B-dense do not fit 32 GB at usable quant.

Max concurrent-agent capacity (32 GB, target ≥30 tok/s each): - **Qwen3-Coder-30B-A3B, Q4/AWQ (~18 GB weights)** → ~12–14 GB free for KV → **12+ concurrent 16k sequences** (q8 KV) or **~8 concurrent 32k**. This is the recommended config and comfortably covers 6–12 agents. - **Qwen2.5-Coder-32B / Qwen3-32B, Q4 (~22 GB weights)** → ~8–9 GB free for KV → **~4 concurrent 16k** (q8 KV) or **~2 concurrent 32k**. Dense 32B is the concurrency bottleneck; keep it for low-parallelism heavy-coding lanes.

2. Ranked model table

Weights VRAM = 4-bit quant unless noted. tok/s = single-stream decode on RTX 5090 (32 GB) unless the source is flagged. Sources in §8.

Rank	Model	Params (active)	Rec. quant	Weights VRAM	Native ctx	Single-stream tok/s (5090)	Comments
1	Qwen3-Coder-30B-A3B-Instruct	30.5B MoE / 3.3B active , 128 experts (8 active)	Q4_K_XL (llama.cpp) or AWQ (vLLM)	~18 GB	256K (→ 1M YaRN)	~ 140 [awesomeagents, measured]	Strong agent, suitable for AI dev. ~61% max low (UN) exact
2	Qwen2.5-Coder-32B-Instruct	32B dense	Q4_K_M	~22 GB	32K (128K YaRN)	~ 45 [model fit, est] / ~58 analogue [Qwen3-32B measured]	Hu 88. Mu Liv 51.
3	Devstral-Small-2507	24B dense (from Mistral-Small-3.1)	Q4_K_M	~15 GB	128K	~55–65 (24B class, est)	SW Ver (#1 Jul)
4	Qwen3-32B	32B dense	Q4_K_M	~22.2 GB	32K (128K YaRN)	~ 58 [awesomeagents, measured]	Strong gen
5	Gemma 3 27B-it	27B dense	Q4_K_M	~22.5 GB	128K	~ 62 [measured]	Good we coc
—	GLM-4.5-Air		—		128K	n/a	

Rank	Model	Params (active)	Rec. quant	Weights VRAM	Native ctx	Single-stream tok/s (5090)	Co
		106B MoE / 12B active		~60 GB Q4			BF TA (str
—	Llama-3.3-70B	70B dense	Q2–Q3	~40–46 GB	128K	~23–35 [est]	me Q2/
—	GLM-4.7 / GLM-5 / Qwen3.5 / Qwen3.6 / DeepSeek V4	—	—	—	—	aggregator claims (e.g. “151 tok/s”)	UN

Coding-quality ordering (published benches): Qwen2.5-Coder-32B $\geq$ Qwen3-32B $\approx$ Qwen3-Coder-30B-A3B > Devstral-Small-2507 > Gemma-3-27B. **Agentic/tool-use ordering:** GLM-4.5-Air (doesn’t fit) > Qwen3-32B (BFCL 75.7) $\approx$ Qwen3-Coder-30B-A3B > Devstral (SWE-agent specialist) > Qwen2.5-Coder-32B. **Throughput-under-concurrency ordering:** Qwen3-Coder-30B-A3B (MoE) $\gg$ Devstral-24B > Qwen3-32B $\approx$ Qwen2.5-Coder-32B.

3. Serving-infra decision matrix (RTX 5090 / Blackwell sm_120)

Engine	Concurrency throughput	VRAM efficiency	Blackwell sm_120 status (2025 – 2026)	Ease of 5090
vLLM	Best. Continuous batching + PagedAttention: ~793 TPS aggregate vs	Best (paged KV, no per-slot waste)	Works, but source build required. torch 2.9 cu128 (nightly), TORCH_CUDA_ARCH_LIST="12.0", VLLM_FLASH_ATTN_VERSION=2 (FA3	Medium (one-time build/Docker)

Engine	Concurrency throughput	VRAM efficiency	Blackwell sm_120 status (2025 → 2026)	Ease of use on 5090
	Ollama 41 TPS at load ; 4 parallel = 341 TPS vs Ollama 123.		unsupported on Blackwell). Prebuilt wheels fail.	
llama.cpp (llama-server)	Good, not great. Slot batching ~3.8× over sequential; scales worse than vLLM at high parallel.	Good with q8 KV + -fa; per-slot KV split from one --ctx-size budget.	Easiest. CUDA backend builds cleanly for sm_120 (-DCMAKE_CUDA_ARCHITECTURES=120); GGUF quant selection is trivial.	Easiest
SGLang	Excellent on supported HW (NVFP4 MoE up to 4× Hopper).	Excellent	Risky on 5090: FP8-blockwise not supported on sm_120; users hit “no kernel image available” / CUDA-graph RMSNorm failures. NVFP4 partial.	Hard
TensorRT-LLM	Highest raw perf on datacenter Blackwell (B200/GB200/RTX PRO 6000) with NVFP4.	Excellent (NVFP4)	NVFP4 gated to SM10x in trtllm-gen; SM120 (5090) NVFP4 KV not wired (open issue #10241).	Hard
Ollama	Poor under load — degrades badly at 16+ concurrent; ~41 TPS at peak vs vLLM 793.	ok single-user	Works (llama.cpp under the hood).	Easiest

4. Concurrency / VRAM math (the load-bearing section)

KV-cache per token = $2 \text{ (K+V)} \times n_layers \times n_kv_heads \times head_dim \times bytes_per_elem$. $bytes_per_elem = 2 \text{ (fp16)} / 1 \text{ (q8_0)} / 0.5 \text{ (q4)}$. Quantised KV requires flash attention (-fa).

Qwen2.5-Coder-32B / Qwen3-32B (dense, GQA: 64 layers, 8 KV heads, head_dim 128)

Per-token: $fp16 = 2 \times 64 \times 8 \times 128 \times 2 = 256 \text{ KB/tok}$; $q8_0 = 128 \text{ KB/tok}$.

Context	fp16 KV / seq	q8_0 KV / seq
8k	2.0 GB	1.0 GB
16k	4.0 GB	2.0 GB
32k	8.0 GB	4.0 GB

Weights (Q4) $\approx 22 \text{ GB} \rightarrow \sim 8\text{--}9 \text{ GB usable for KV}$ (leave $\sim 1.5 \text{ GB}$ headroom on 32 GB). $\rightarrow$ q8 KV: ~ 4 concurrent @16k, ~ 2 concurrent @32k, ~ 8 @8k. Dense 32B cannot serve 6–12 agents at 32k.

Qwen3-Coder-30B-A3B (MoE — 48 layers, ~ 4 KV heads, head_dim 128; head count approximate/UNCONFIRMED exact)

Per-token (approx): $fp16 \approx 2 \times 48 \times 4 \times 128 \times 2 = 96 \text{ KB/tok}$; $q8_0 \approx 48 \text{ KB/tok}$ — roughly $\frac{1}{3}$ of the dense 32B.

Context	fp16 KV / seq	q8_0 KV / seq
8k	$\sim 0.75 \text{ GB}$	$\sim 0.38 \text{ GB}$
16k	$\sim 1.5 \text{ GB}$	$\sim 0.75 \text{ GB}$
32k	$\sim 3.0 \text{ GB}$	$\sim 1.5 \text{ GB}$

Weights (Q4) $\approx 18 \text{ GB} \rightarrow \sim 12\text{--}13 \text{ GB usable for KV}$. $\rightarrow$ q8 KV: $\sim 12\text{--}16$ concurrent @16k, ~ 8 concurrent @32k, many @8k. This is why the MoE model is the concurrency answer.

Note on aggregator error: one source quoted “FP16 KV = 32 GB for a 32B @32K.” That does not match GQA (8 KV-head) math (8 GB) — likely a non-GQA or different-model config. Use the computed table above.

5. Exact launch commands

5A. vLLM — PRIMARY (max concurrency), Qwen3-Coder-30B-A3B AWQ

Build once for sm_120 (host or container):

```
uv venv --python 3.12 --seed && source .venv/bin/activate
pip install --pre torch --index-url https://download.pytorch.org/
    whl/nightly/cu128 # torch 2.9.x+cu128
git clone https://github.com/vllm-project/vllm && cd vllm
python use_existing_torch.py
pip install -r requirements/build.txt
export VLLM_FLASH_ATTN_VERSION=2 TORCH_CUDA_ARCH_LIST="12.0"
    MAX_JOBS=6
pip install --no-build-isolation -e .
# Requires: NVIDIA driver >=575.64.03, CUDA 12.8, Ubuntu 24.04+
```

Serve (single 5090, ~12 agents):

```
vllm serve Qwen/Qwen3-Coder-30B-A3B-Instruct-AWQ \
    --tensor-parallel-size 1 \
    --gpu-memory-utilization 0.90 \
    --max-model-len 32768 \
    --max-num-seqs 12 \
    --enable-prefix-caching \
    --enable-auto-tool-choice --tool-call-parser qwen3_coder
```

Notes: --enable-prefix-caching is a big win for agents (shared system/tool preamble reused). If WSL2 or CUDA-graph instability appears, add --enforce-eager (costs ~40% decode — Linux bare-metal usually doesn't need it). Drop --gpu-memory-utilization to 0.50 if you must co-locate another GPU process (vLLM's default 0.9 pre-allocation starves neighbours).

5B. llama.cpp llama-server — FALLBACK (easiest on Blackwell)

Build:

```
cmake -B build -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=120
cmake --build build --config Release -j
```

Serve Qwen3-Coder-30B-A3B, 12 slots @ ~16k each (192k total KV budget, q8 KV):

```
./build/bin/llama-server \
    -hf unsloth/Qwen3-Coder-30B-A3B-Instruct-GGUF:Q4_K_XL \
    -ngl 99 --jinja \
```

```
--parallel 12 --cont-batching \
--ctx-size 196608 \
    # total KV split across 12 slots => ~16k/slot
-fa on --cache-type-k q8_0 --cache-type-v q8_0 \
--temp 0.7 --top-p 0.8 --top-k 20 --repeat-penalty 1.05 \
--host 0.0.0.0 --port 8080
```

Key facts: `--ctx-size` is the **total** KV budget shared by `--parallel` slots (not per-request). `-fa on` is mandatory for q8 KV. Sampling values are Qwen's recommended settings.

Dense heavy-coding lane (Qwen2.5-Coder-32B, low parallelism):

```
./build/bin/llama-server -hf bartowski/Qwen2.5-Coder-32B-
    Instruct-GGUF:Q4_K_M \
--ngl 99 --jinja --parallel 4 --cont-batching \
--ctx-size 65536 -fa on --cache-type-k q8_0 --cache-type-v
    q8_0 --port 8081
```

6. Optimization tricks (ranked by payoff here)

1. **Flash attention (-fa / FA2 on vLLM)**. 1.3–2× faster prefill + enables KV quant. FA3 is **not** on Blackwell yet → pin FA2 (VLLM_FLASH_ATTN_VERSION=2).
2. **q8_0 KV cache** (`--cache-type-k/v q8_0`). Halves KV VRAM vs fp16 with negligible quality loss → directly doubles concurrent-sequence capacity. (q4 KV = risky, more quality loss — avoid for coding.)
3. **Prefix / prompt caching** (vLLM `--enable-prefix-caching`; llama.cpp reuses slot prefix). Agents resend a large fixed system+tool preamble every turn → cache turns that into near-zero prefill. High payoff for CLI agents.
4. **Continuous / in-flight batching** (vLLM default; llama.cpp `--cont-batching`). The single biggest concurrency lever — CUDA graphs account for ~65–77% of vLLM's advantage; keep them on (don't `--enforce-eager` unless forced).
5. **AWQ / GPTQ 4-bit + Marlin kernels** (vLLM 0.19+ has AWQ-Marlin). On a 1.79 TB/s bus, 4-bit is a mild quality trade for big VRAM savings; note one Blackwell benchmark found 4-bit *slower* than BF16 for sub-20B models (bandwidth already ample) — so 4-bit is about **fitting weights + KV headroom**, not raw speed, for 30–32B.
6. **Speculative decoding — mostly skip**. Recent llama.cpp benchmarks (incl. Blackwell RTX 5060 Ti, Ampere) show it is **net-negative or neutral for MoE/A3B models** and only wins in a narrow corner. vLLM MTP/EAGLE-3 can help dense models but is fiddly. Do **not** rely on it for the MoE primary.

7. NVFP4 (native Blackwell 4-bit) is promising but **not yet wired for sm_120 consumer cards** in TensorRT-LLM/SGLang — revisit later.

7. Rootless Podman + GPU passthrough (CUDA Blackwell)

Prereqs: recent NVIDIA driver (≥575.x for sm_120), nvidia-container-toolkit, Podman ≥4.1 (CDI in --device).

```
# 1. Generate CDI spec (rootless: write to user dir; else /etc/
    cdi needs root once)
nvidia-ctk cdi generate --output=$HOME/.config/cdi/nvidia.yaml
nvidia-ctk cdi list      # verify nvidia.com/gpu=all / =0 present

# 2. Smoke test (rootless)
podman run --rm \
    --device nvidia.com/gpu=all \
    --security-opt=label=disable \
    nvcr.io/nvidia/cuda:12.8.0-base-ubuntu24.04 nvidia-smi

# 3. If GPU device access is group-gated:
podman run --rm --device nvidia.com/gpu=all --group-add keep-
    groups ...
```

Notes / caveats: - **Base image:** use CUDA 12.8 (nvcr.io/nvidia/cuda:12.8.0-*ubuntu24.04) — 12.8 is the first toolkit with sm_120 (nvcc -arch=sm_120). Older 12.x images cannot compile Blackwell kernels. - Rootless CDI works but is the fragile path: there are documented cases (containers/podman #17539) where CDI devices resolve under root/privileged but not rootless. If nvidia-smi fails rootless, confirm the user can read the CDI spec dir and has device-file access (ls -l /dev/nvidia*), and pass --cdi-spec-dir=\$HOME/.config/cdi explicitly. - For vLLM, run inside a CUDA 12.8 image, build vLLM as in §5A, then podman run --device nvidia.com/gpu=all -p 8000:8000 <image> vllm serve - Per project constitution (§11.4.76 / §11.4.161): drive containers through the vasic-digital/containers submodule / rootless Podman — the commands above are the underlying mechanism, not a bypass.

8. Sources verified 2026-07-06

- vLLM vs Ollama on Blackwell (throughput 793 vs 41 TPS, VRAM, CUDA-graph impact, launch flags) — <https://allenkuo.medium.com/vllm-or-ollama-on->

blackwell-benchmarks-landmines-and-what-agents-actually-need-5dc539bb28ef

- RTX 5090 per-model tok/s + KV estimates (model fit; explicitly estimates, not measured) — <https://model fit.io/gpu/rtx-5090/>
- Home GPU LLM Leaderboard (measured 5090 tok/s: Qwen3-32B ~58, Qwen3-Coder-30B-A3B ~140, Llama-3.3-70B ~35, Gemma-3-27B ~62) — <https://awesomeagents.ai/leaderboards/home-gpu-llm-leaderboard/>
- Qwen2.5-Coder family blog (HumanEval 88.4, MBPP 84.0, MultiPL-E 75.4, LiveCodeBench 51.2, Aider 73.7) — <https://qwenlm.github.io/blog/qwen2.5-coder-family/>
- Qwen2.5-Coder Technical Report — <https://arxiv.org/html/2409.12186v3>
- Qwen3-Coder-30B-A3B GGUF + tool-calling fixes — <https://huggingface.co/unsloth/Qwen3-Coder-30B-A3B-Instruct-GGUF>
- Qwen3-Coder run-locally (llama.cpp flags, sampling temp 0.7/top_p 0.8/top_k 20/rep 1.05, ctx) — <https://unsloth.ai/docs/models/tutorials/qwen3-coder-how-to-run-locally>
- Qwen3-Coder MoE spec (30.5B/128 experts/8 active/256K ctx) + Aider-Polyglot note — <https://openrouter.ai/qwen/qwen3-coder-30b-a3b-instruct>
- Devstral-Small-2507 (24B, SWE-bench Verified 53.6%, #1 open, 128K, RTX 4090-class) — <https://mistral.ai/news/devstral-2507/>, <https://huggingface.co/mistralai/Devstral-Small-2507>
- Berkeley Function Calling Leaderboard (BFCL) — GLM-4.5 76.7, Qwen3-32B 75.7 (v3) — <https://gorilla.cs.berkeley.edu/leaderboard.html>
- GLM-4.5 paper (GLM-4.5-Air BFCL v3 76.4, TAU-retail 77.9; 106B/12B active) — <https://arxiv.org/pdf/2508.06471>
- vLLM on RTX 5090 working setup (torch 2.9 cu128, TORCH_CUDA_ARCH_LIST=12.0, FA2, source build, driver 575.64.03) — <https://discuss.vllm.ai/t/vllm-on-rtx5090-working-gpu-setup-with-torch-2-9-0-cu128/1492>
- vLLM sm_120 feature/tracking issue — <https://github.com/vllm-project/vllm/issues/13306>; steps issue — <https://github.com/vllm-project/vllm/issues/14452>
- SGLang Blackwell sm_120 status (FP8-blockwise unsupported, kernel-image errors, NVFP4 partial) — <https://github.com/sgl-project/sglang/discussions/9543>, <https://github.com/sgl-project/sglang/issues/9233>
- TensorRT-LLM NVFP4 SM120 gap (open issue) — <https://github.com/NVIDIA/TensorRT-LLM/issues/10241>; TRT-LLM overview — <https://nvidia.github.io/TensorRT-LLM/>
- llama.cpp server README (--parallel, --cont-batching, --cache-type-k/v, -fa, KV types incl q8_0) — <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>
- llama.cpp batching/--ctx-size semantics — <https://github.com/ggml-org/llama.cpp/discussions/4130>

- llama.cpp speculative decoding (net-negative for A3B/MoE on consumer GPUs) — <https://github.com/thc1006/qwen3.6-speculative-decoding-rtx3090> , <https://github.com/ggml-org/llama.cpp/blob/master/docs/speculative.md>
- KV-cache calculation formula + q8/fp16/q4 bytes — <https://lyceum.technology/magazine/kv-cache-memory-calculation-llm/> , <https://www.techplained.com/kv-cache-quantization>
- NVIDIA Container Toolkit CDI support (nvidia-ctk cdi generate, --device nvidia.com/gpu=all, rootless notes) — <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/cdi-support.html>
- Podman GPU access + rootless --group-add keep-groups — <https://podman-desktop.io/docs/podman/gpu> ; rootless CDI issue — <https://github.com/containers/podman/issues/17539>

UNCONFIRMED (excluded from recommendations): GLM-4.7 / GLM-5 / Qwen3.5 / Qwen3.6 / DeepSeek V4 tok-s and benchmark claims from aggregator sites (modelfit, spheron, insiderllm, dubesor) — no primary model card or reproducible benchmark located as of 2026-07-06; treat as speculative until a first-party source appears.